SOARASPBERRY

SOARASPBERRY
- 1. Descripción
- 2. Configuración
  - 2.1 Configuración estática
    - 2.1.1 Puerto TCP/IP
    - 2.1.2 Configuración Dali
    - 2.1.3 Configuración de puertos
    - 2.1.4 Configuración de ficheros
    - 2.1.5 Configuración de logs
  - 2.2 Configuración dinámica
    - 2.2.1 Configuración reles
    - 2.2.2 Configuración entradas
    - 2.2.3 Configuración reles conmutados
- 3. Funciones
  - programa()
  - signalHandlesConfig()
  - handler()
    - Parámetros
  - puertoSerieConfig()
  - TRB_ed()
    - Parámetros
  - TRB_ea()
    - Parámetros
  - TRB_sd()
    - Parámetros
  - TRB_sa()
    - Parámetros
  - TRB_dali()
    - Parámetros
  - TRB_sht31()
    - Parámetros
  - TRB_scd30()
    - Parámetros
  - TRB_ds18b20()
    - Parámetros
  - TRB_pulsos()
    - Parámetros
  - TRB_velocidad_viento()
    - Parámetros
  - obtenerWPI_entrada()
    - Parámetros
    - Return
  - obtenerWPI_salida()

Parámetros
Return
set_salida_rele()
Parámetros
cambiar_rele()
Parámetros
conmutar_rele()
Parámetros
motor_lector_entradas_digitales()
motor_lector_velocidad_viento()
lector_entradas_analogicas()
contador()
get_entrada_digital()
Parámetros
Return
get_entrada_analogica()
Parámetros
Return
get_estado_rele()
Parámetros
Return
get_contador()
Parámetros
Return
get_estado_reles()
set_salida()
Parámetros
velocidad_viento_activo()
guardar_estado_salidas()
leer_estado_salidas()
leer_configuracion()
writePid()
4. Compilación

1. Descripción

Este software esta diseñado para ejecutarse en una `Raspberry Pi` (cualquier modelo) con sistema operativo `Raspbian`, instalada sobre la `Placa de Control y Potencia V1.0`, con el `Adaptador Raspberry V1.0` ó `V1.1`. Es capaz de controlar todos los dispositivos de la placa.

> Es importante destacar que este software se ejecuta sobre un sistema operativo, y por ello necesita cierto mantenimiento.

2. Configuración

2.1 Configuración estática

Esta es la configuración del sistema definida en el archivo `source.h`. Si se modifica esta configuración, será necesario volver a compilar.

2.1.1 Puerto TCP/IP

- `PUERTO_ESCUCHA`: Puerto al que la raspberry escucha peticiones. Por defecto **`9000`**.

2.1.2 Configuración Dali

- `CONF_DALI_VELOCIDAD`: Velocidad del bus dali. Por defecto **`38400`**.
- `CONF_DALI_PARIDAD`: Configuración de paridad. Por defecto **`0`**.
- `CONF_DALI_BITDATOS`: Configuración de bits de datos. Por defecto **`8`**.
- `CONF_DALI_BITSTOP`: Configuración de bits de stop. Por defecto **`1`**.

2.1.3 Configuración de puertos

- `RUTA_PUERTO_I2C`: Puerto I2C. Por defecto **`/dev/i2c-1`**.
- `RUTA_PUERTO_SERIE_DALI_1`: Puerto Dali 1. Por defecto **`/dev/ttyDALI1`**.
- `RUTA_PUERTO_SERIE_DALI_2`: Puerto Dali 2. Por defecto **`/dev/ttyDALI2`**.
- `RUTA_PUERTO_SERIE_DALI_3`: Puerto Dali 3. Por defecto **`/dev/ttyDALI3`**.
- `RUTA_PUERTO_SERIE_DALI_4`: Puerto Dali 4. Por defecto **`/dev/ttyDALI4`**.
- `RUTA_PUERTO_SERIE`: Puerto I2C. Por defecto **`/dev/ttyAMA0`**.

2.1.4 Configuración de ficheros

- `FICHERO_PID`: Ruta del fichero donde se guarda el PID. Por defecto **`/var/run/soaraspberry.pid`**.
- `FICHERO_DATOS`: Ruta del fichero donde se guardan los datos necesarios del sistema. Por defecto **`/home/soa/soaraspberry/soa.data`**.
- `FICHERO_CONFIGURACION`: Ruta del fichero de donde se lee la configuración del sistema. Por defecto **`/home/soa/soaraspberry/soa.conf`**.

2.1.5 Configuración de logs

- `FICHERO_LOG_TRACE`: Ruta del fichero donde se escriben logs de traza. Por defecto **`/home/soa/soaraspberry/logs/trace.log`**.
- `FICHERO_LOG_DEBUG`: Ruta del fichero donde se escriben logs de depuración. Por defecto **`/home/soa/soaraspberry/logs/debug.log`**.
- `FICHERO_LOG_INFO`: Ruta del fichero donde se escriben logs de información. Por defecto **`/home/soa/soaraspberry/logs/info.log`**.
- `FICHERO_LOG_WARN`: Ruta del fichero donde se escriben logs de aviso. Por defecto **`/home/soa/soaraspberry/logs/warn.log`**.
- `FICHERO_LOG_ERROR`: Ruta del fichero donde se escriben logs de error. Por defecto **`/home/soa/soaraspberry/logs/error.log`**.
- `FICHERO_LOG_FATAL`: Ruta del fichero donde se escriben logs de errores fatales. Por defecto **`/home/soa/soaraspberry/logs/fatal.log`**.

2.2 Configuración dinámica

Estas son las variables que hay que configurar para que la placa funcione adecuadamente. Es importante que esta configuración sea correcta, ya que de lo contrario, obtendremos un funcionamiento inesperado. Hay que tener en cuenta que cada placa lleva una configuración distinta. Todas estas variables de configuración se encuentran en el fichero `soa.conf`.

2.2.1 Configuración reles

En la sección de `RELAY_CONFIG`:

`_CONFIG_BORNA_SALIDA_1`, `_CONFIG_BORNA_SALIDA_2`, `_CONFIG_BORNA_SALIDA_3`, `_CONFIG_BORNA_SALIDA_4`, `_CONFIG_BORNA_SALIDA_5`, `_CONFIG_BORNA_SALIDA_6`, `_CONFIG_BORNA_SALIDA_7`, `_CONFIG_BORNA_SALIDA_8`, `_CONFIG_BORNA_SALIDA_9`, `_CONFIG_BORNA_SALIDA_10`, `_CONFIG_BORNA_SALIDA_11`, `_CONFIG_BORNA_SALIDA_12`, `_CONFIG_BORNA_SALIDA_13`, `_CONFIG_BORNA_SALIDA_14`, `_CONFIG_BORNA_SALIDA_15`, `_CONFIG_BORNA_SALIDA_16`

Estas variables tienen tres configuraciones posibles:

- `0` - Simple: el valor que llega es el que se escribe
- `1` - Cambio: llegue el valor que llege siempre cambia su estado al contrario
- `2` - Conmutado: Se hará la salida en funcion de una entrada y de el valor recibido

> Para la configuración `2` (Conmutado), véase la `sección 2.4`

2.2.2 Configuración entradas

En la sección `INPUT_CONFIG`

`_CONFIG_BORNA_ENTRADA_1`, `_CONFIG_BORNA_ENTRADA_2`, `_CONFIG_BORNA_ENTRADA_3`, `_CONFIG_BORNA_ENTRADA_4`, `_CONFIG_BORNA_ENTRADA_5`, `_CONFIG_BORNA_ENTRADA_6`, `_CONFIG_BORNA_ENTRADA_7`, `_CONFIG_BORNA_ENTRADA_8`, `_CONFIG_BORNA_ENTRADA_9`, `_CONFIG_BORNA_ENTRADA_10`, `_CONFIG_BORNA_ENTRADA_11`, `_CONFIG_BORNA_ENTRADA_12`, `_CONFIG_BORNA_ENTRADA_13`, `_CONFIG_BORNA_ENTRADA_14`, `_CONFIG_BORNA_ENTRADA_15`, `_CONFIG_BORNA_ENTRADA_16`

Estas variables tienen tres configuraciones posibles:

- `0` - Simple: entrada normal
- `1` - Contador: entrada contador de pulsos
- `2` - Contador: entrada contador velocidad del viento

2.2.3 Configuración reles conmutados

En la sección `INPUT_RELAY_REL`

`_REL_BORNA_SALIDA_1`, `_REL_BORNA_SALIDA_2`, `_REL_BORNA_SALIDA_3`, `_REL_BORNA_SALIDA_4`, `_REL_BORNA_SALIDA_5`, `_REL_BORNA_SALIDA_6`, `_REL_BORNA_SALIDA_7`, `_REL_BORNA_SALIDA_8`, `_REL_BORNA_SALIDA_9`, `_REL_BORNA_SALIDA_10`, `_REL_BORNA_SALIDA_11`, `_REL_BORNA_SALIDA_12`, `_REL_BORNA_SALIDA_13`, `_REL_BORNA_SALIDA_14`, `_REL_BORNA_SALIDA_15`, `_REL_BORNA_SALIDA_16`

Para las bornas (reles) configuradas como conmutadas (`2`), hay que indicar la entrada a la que tienen que leer antes de escribir. Los posibles valores son de `0 a 15`, por defecto, si no es una conmutada, el valor debe ser `255`.

3. Funciones

programa()

```
void programa();
```

Proceso principal, configura señales, configura el puerto TCP/IP para recibir peticiones, configura los puertos serie, configura entrada/salida, inicia el hilo `motor_lectura_entradas()` que lee las entradas y crea un bucle que gestiona las peticiones entrantes. El bucle solo finaliza su ejecucion si recibe alguna de estas señales: `SIGTERM`, `SIGINT`, `SIGHUP`. Cuando esto pasa, cierra el puerto de escucha, espera 3 segundos para que los hilos que se esten ejecutando, terminen correctamente su ejecución y cierra los puertos serie. Después de esto, el programa acaba.

---

signalHandlesConfig()

```
void signalHandlerConfig();
```

Configura el menejo de señales de sistema.

---

handler()

```
void handler(int signal);
```

Dada una señal, ejecuta las acciones apropiadas.

Parámetros

signal (int) señal recibida

---

puertoSerieConfig()

```
void puertoSerieConfig();
```

Abre los puertos serie.

TRB_ed()

```
void *TRB_ed(void *datos_trama);
```

Lee las entradas digitales indicadas en la trama recibida y envia una trama con el valor de cada entrada.

Parámetros

datos_trama (struct TRB)

TRB_ea()

```
void *TRB_ea(void *datos_trama);
```

Lee las entradas analógicas indicadas en la trama recibida y envia una trama con el valor de cada entrada.

Parámetros

datos_trama (struct TRB)

TRB_sd()

```
void *TRB_sd(void *datos_trama);
```

Realiza las salidas digitales indicadas en la trama de entrada.

Parámetros

datos_trama (struct TRB)

TRB_sa()

```
void *TRB_sa(void *datos_trama);
```

Realiza las salidas analógicas indicadas en la trama de entrada.

Parámetros

datos_trama (struct TRB)

TRB_dali()

```
void *TRB_dali(void *datos_trama);
```

Lee la trama dali de la trama recibida y la escribe en el bus dali, espera una respuesta y envia una trama con la trama de respuesta del dali.

Parámetros

datos_trama (struct TRB)

TRB_sht31()

```
void *TRB_sht31(void *datos_trama);
```

Lee las sondas SHT31 indicadas en la trama recibida y envia una trama con el valor de cada sonda.

Parámetros

datos_trama (struct TRB)

TRB_scd30()

```
void *TRB_scd30(void *datos_trama);
```

Lee las sondas de CO2 SCD30 indicadas en la trama recibida y envia una trama con el valor de cada sonda.

Parámetros

datos_trama (struct TRB)

TRB_ds18b20()

```
void *TRB_ds18b20(void *datos_trama);
```

Lee las sondas DS18B20 indicadas en la trama recibida y envia una trama con el valor de cada sonda.

Parámetros

datos_trama (struct TRB)

TRB_pulsos()

```
void *TRB_pulsos(void *datos_trama);
```

Lee el contador de pulsos de cada entrada indicada en la trama recibida y envia una trama con el valor de cada entrada.

Parámetros

datos_trama (struct TRB)

TRB_velocidad_viento()

```
void *TRB_velocidad_viento(void *datos_trama);
```

Lee el contador de pulsos de velocidad del viento de cada entrada indicada en la trama y envia una trama con el valor en km/h de cada entrada.

Parámetros

datos_trama (struct TRB)

---

obtenerWPI_entrada()

```
int obtenerWPI_entrada(int borna);
```

Solo para adaptador raspberry V1.

Esta funcion devuelve el pin de la raspberry equivalente a una borna de entrada.

Parámetros

borna (int) borna digital [0 - 15]

Return

borna WPI equivalente

---

obtenerWPI_salida()

```
int obtenerWPI_salida(int borna);
```

Solo para adaptador raspberry V1.

Esta funcion devuelve el pin de la raspberry equivalente a una borna de salida.

Parámetros

borna (int) borna digital [0 - 15]

Return

borna WPI equivalente

---

set_salida_rele()

```
void set_salida_rele(char borna, char value);
```

Ejecuta el cambio de estado de un rele, en funcion de su configuracion.

Parámetros

borna (char) borna de salida que queremos cambiar

value (char) valor que queremos que obtenga

cambiar_rele()

```
void cambiar_rele(char rele);
```

Cambia el estado de un rele, si esta a 0 lo pone a 1 y viceversa.

Parámetros

rele (char) número de rele que queremos cambiar

conmutar_rele()

```
void conmutar_rele(char rele, char valor);
```

Conmuta el estado de un rele. Para que esta funcion funcione adecuadamente, es importante que el rele esta bien configurado. Es necesario configurar que entrada esta asociada a dicha salida.

Parámetros

rele (char) número de rele que queremos conmutar

valor (char) valor que queremos que obtenga

motor_lector_entradas_digitales()

```
void *motor_lector_entradas_digitales();
```

Motor que lee constantemente el estado de las entradas digitales en un intervalo de tiempo de 10ms y las guarda en una varible global.

motor_lector_velocidad_viento()

```
void *motor_lector_velocidad_viento();
```

Motor que lee constantemente el estado de las entradas digitales en un intervalo de tiempo de `TIEMPO_MUESTREO_VELOCIDAD_VIENTO` y las guarda en una variable global. Este motor solo se ejecuta si alguna entrada ha sido configurada como contador de velocidad del viento `2`.

---

lector_entradas_analogicas()

```
void lector_entradas_analogicas();
```

Lee las entradas analógicas y las guarda en una variable global.

---

contador()

```
void contador();
```

Esta funcion cuenta la cantidad de veces que una entrada cambia de 0 a 1, se ejecuta dentro de `*motor_lector_entradas_digitales()`.

---

get_entrada_digital()

```
int get_entrada(char borna);
```

Devuelve el estado de una entrada digital.

Parámetros

borna (char) borna que queremos leer

Return

valor de la borna requerida

---

get_entrada_analogica()

```
int get_entrada_analogica(char borna);
```

Devuelve el valor de una entrada analógica

Parámetros

borna (char) borna que queremos leer

Return

valor de la borna requerida

---

get_estado_rele()

```
int get_estado_rele(char borna);
```

Devuelve el estado de un rele.

Parámetros

borna (char) borna que queremos leer

Return

valor de la borna requerida

---

get_contador()

```
unsigned long int get_contador(char borna);
```

Devuelve la cuenta de un contador.

Parámetros

borna (char) contador que queremos leer

Return

valor del contador requerido

---

get_estado_reles()

```
void get_estado_reles();
```

Hace una lectura del estado de los reles y los guarda en una variable global.

---

set_salida()

```
void set_salida(char borna, char value, unsigned char guardar);
```

Ejecuta una salida.

Parámetros

borna (char) contador que queremos cambiar

value (char) valor que queremos que obtenga

guardar (unsigned char) si esta variable es `TRUE` guarda la salida que acabamos de hacer en la variable global del estado de las salidas.

---

velocidad_viento_activo()

```
unsigned char velocidad_viento_activo();
```

Comprueba si alguna borna esta configurada como contador de velocidad del viento `2`. Usamos esta funcion para saber si tenemos que iniciar el motor `*motor_lector_velocidad_viento()`.

---

guardar_estado_salidas()

```
void guardar_estado_salidas();
```

Cada vez que se ejecuta una salida, guardamos el estado de todas las salidas en un fichero para tener el estado si se reinicia.

---

leer_estado_salidas()

```
void leer_estado_salidas();
```

Lee el estado de las salidas del fichero y actualiza las variables internas.

---

leer_configuracion()

```
void leer_configuracion();
```

Lee la configuración del sistema.

---

writePid()

```
void writePid();
```

Escribe el PID del proceso en el fichero correspondiente.

---

4. Compilación

El `Makefile` permite compilar el sistema de diferentes maneras, que haran que se comporte de forma distinta:

- Compilar el proyecto en modo depuración mostrando todos los mensajes por pantalla

```
$ make debug
```

- Compilar el proyecto en modo producción para ejecutarlo como demonio

```
$ make prod
```

- Compilar el proyecto en modo depuración mostrando todos los mensajes por pantalla pero obviando aquellos mensajes que salen demasiado rápido, como las entradas digitales.

```
$ make custom MODO=LIMITED_DEBUG
```